

Multi-Source Candidate Data Transformer — Technical Design

Eklavya Sharma · eklavya.sharma.ug23@nsut.ac.in · Eightfold Engineering Intern (Jul–Dec 2026), Stage 2

Pipeline

I broke the pipeline into seven stages: **detect** → **extract** → **normalize** → **merge** → **confidence** → **project** → **validate**. Detect just figures out, from the manifest, which extractor each source needs. Extract pulls a raw, not-yet-normalized record out of each source — this is also where ATS's own field names get remapped onto the fields I actually use — and is wrapped so that a bad source produces a logged error rather than a crash. Normalize converts each raw field into a standard format. Merge combines the normalized values across sources into one record and resolves any conflicts along the way. Confidence scores every field that ended up populated. Project reshapes the canonical record according to whatever runtime config was passed in. Validate is a last check on the projected output's shape before it gets returned.

Canonical Schema & Normalized Formats

The default schema follows the brief's table (candidate_id, full_name, emails[], phones[], location, links, headline, years_experience, skills[], experience[], education[], provenance[], overall_confidence). I added one extra field, **source_errors**, which logs which sources failed and why for a given candidate — it answers "why is this profile thin?" the same way provenance answers "why is this particular value what it is?"

- **Phones** normalize to E.164 using the *phonenumbers* library. If there's no country code and no location hint to infer one from, it falls back to a default region but flags the result as a guess (lower confidence) rather than treating it like a clean parse.
- **Dates** become YYYY-MM. A year-only value like "2019" becomes 2019-01 but with an explicit low-precision note attached — I didn't want a default month to look any more certain than it actually is. "Present"/"Current" are treated as open-ended (null end date) instead of being forced into a real date.
- **Country** normalizes to ISO 3166-1 alpha-2 using *pycountry* plus a small alias table. Two-letter tokens get checked against US state abbreviations before the country lookup runs, so something like "San Francisco, CA" doesn't get misread as Canada.
- **Skills** map to a canonical name through an alias table ("js"/"JS" → JavaScript). Anything not in the table gets kept as-is, just with reduced confidence, rather than dropped.
- **Names** only get trimmed and have whitespace collapsed — never re-cased. Auto-titlecasing breaks real names (think "O'Brien"), and that felt like exactly the kind of mistake worth avoiding on purpose.

Merge & Conflict-Resolution Policy

Match key: I'm assuming an upstream manifest already supplies the crosswalk — which CSV row, ATS record, GitHub fixture, and notes file all belong to a given candidate_id. More on why under Out of Scope.

Scalar fields (name, headline, years_experience, location) are winner-take-all, decided by a per-field source-priority table — ATS beats CSV beats GitHub beats notes for job title, for example, but for skills GitHub and notes outrank ATS and CSV, since actual code is more objective evidence than a recruiter's guess. field_confidence works out to source_weight times normalization_quality, with agreeing sources adding a small bonus (capped at 1.0) and disagreeing ones costing a penalty. A single high-priority source still beats two low-priority ones that happen to agree with each other — it's a hierarchy, not a vote. Whatever loses still gets kept in provenance rather than thrown away.

List fields (emails, phones, skills, experience, education) just get unioned and deduplicated instead of picking a winner — someone can genuinely have two emails, but not two different "true" current titles. **overall_confidence** is the sum of populated-field confidences divided by a fixed field count, so a thin profile scores low instead of looking artificially confident just because the few fields it does have are solid.

Runtime Config (Projection Layer)

The full canonical record always gets built first — the config only acts as a downstream lens on top of it, with no special-casing anywhere. The default schema actually runs through the same projection function as any custom config does, which is what makes "no code changes" true in practice rather than just something I'm claiming. Each field in a config has a *path* (the output key), a *from* (the canonical path it pulls from — supports addressing like emails[0] or skills[].name), a *type*, and optionally *required* and *normalize* (to override the display form of an already-normalized value, e.g. E.164 → national format). A few top-level toggles control the rest: include_confidence (also strips per-skill confidence when off), include_provenance (also gates source_errors), and on_missing, which can be null, omit, or error. error fails loudly for that one candidate specifically — it doesn't take the rest of a batch down with it.

Edge Cases Handled

- Missing or corrupt sources (malformed JSON, a missing file, empty notes) get logged in source_errors and skipped — the candidate still gets a profile built from whatever's left.
- Conflicting values resolve by priority, with the loser kept in provenance along with why it lost.
- Messy dates get a best-effort YYYY-MM parse; year-only values are flagged low-precision instead of guessing a month with false confidence.
- A phone with no country code and no location hint to infer one falls back to a default-region guess that's explicitly flagged, or just null if it still won't validate — nothing's invented.
- The same skill written differently across sources gets canonicalized into a single entry with all contributing sources combined.

What I Left Out (and Why)

Fuzzy cross-source identity resolution when there's no shared ID — matching records across systems without one is its own project, so I pushed that assumption up to the manifest layer instead of trying to half-solve it here. Resume PDF/DOCX parsing — I used GitHub and notes as my two unstructured sources instead, since resume layouts vary enough that doing this properly felt like a separate effort; the extractor interface is the same shape for every source type though, so adding a resume extractor later is one more function, not a redesign. Live GitHub API calls in the test suite — a cached fixture is used instead so results stay deterministic and the tests run offline (the live-fetch code path exists, it just isn't exercised by tests). NER-based skill extraction from free text — dictionary and keyword matching only. And no UI beyond a CLI, since the brief explicitly said that was lower priority.